Comparing Forecasters for Serverless Invocation Provisioning

Mojtaba Jafari
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
mojtaba.jafari@aut.ac.ir

Ehsan Moradi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
your.email@aut.ac.ir

Amir Kalbasi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
kalbasi@aut.ac.ir

***Abstract*—A cold start is the reason to forecast serverless invocations: a request that finds no warm container waits while one is created. The Azure Functions trace used here records per-minute counts, not container starts, so we count the containers a forecast leaves short and the containers it leaves idle, assuming one container serves 10 invocations a minute. Six forecasters and several references are compared on 80 functions in four workload classes, with an hourly 60-minute forecast. A first scoring made Chronos and ETS look wasteful because a forecast of 0.01 invocations still prepares a container. After forecasts below one half are set to zero, their median idle capacity falls from thousands of container-minutes to about a hundred or fewer. A previous-hour copy, seasonal naive, and Holt–Winters have the lowest error and are close to each other, and neither Chronos nor an LSTM beats them. A high quantile of recent load cuts the median shortfall from 75 to 18.5 container-minutes at about 730 extra container-minutes, and a high Chronos quantile does the same. A class-specific choice made on two days was not distinguishable from seasonal naive on the next weekend. A reactive ten-minute keep-alive that reads actual counts had a far lower shortfall than any hourly forecast.**

***Index Terms*—serverless computing, workload forecasting, cold start, provisioning**

I. Introduction

Serverless platforms start a container when a request arrives and none is warm. That cold start is the delay this paper is trying to avoid. Keeping containers warm removes it, and the cost of keeping them is the memory they hold while idle. The Azure Functions trace used below records how many times each function ran in each minute. It does not record container starts, so the shortfall defined later is a stand-in for cold starts, not a count of them.

To reconcile this latency-versus-cost trade-off, cloud platforms increasingly deploy predictive resource provisioning, anticipating future invocation volume to warm containers proactively. Yet, a fundamental disconnect exists between standard time-series evaluation and practical cloud resource management. Traditional statistical metrics, such as symmetric mean absolute percentage error (sMAPE) or mean squared error (MSE), treat forecast deviations symmetrically. In production environments, however, forecasting errors carry profoundly asymmetric consequences: an under-prediction leaves fewer warm containers than the arrival rate needs, which is where a cold start would occur, whereas an over-prediction leaves containers idle. A predictor with the lowest numerical error can still be a poor operating point if its misses are the minutes that need another container.

Compounding this challenge is the extreme heterogeneity inherent in serverless workloads. Large-scale empirical traces, such as those from Microsoft Azure Functions [1], demonstrate that serverless invocations rarely conform to a single distribution. Instead, workloads encompass heavily skewed behaviors: most functions are invoked only a few times a day, some follow a daily shape, and some arrive in bursts. A fleet-wide error hides which forecaster suits which profile.

In this work, we conduct a systematic, capacity-aware empirical comparison of invocation forecasters across representative serverless workload archetypes. Rather than proposing a new autoscaling controller, we measure forecast error beside the container shortfall and the idle capacity that shortfall would have avoided. We evaluate six diverse forecasting strategies—ranging from simple, training-free heuristics (last-value, trailing percentiles, and seasonal naive) to classical time-series smoothing (seasonal Holt–Winters), deep recurrent architectures (LSTM), and modern zero-shot pretrained foundation models (Chronos-Bolt [2])—across 80 diverse functions sampled from the public Azure Functions trace.

This paper compares six forecasters and several references under one capacity assumption, and reports intervals. It makes three points. A continuous forecast needs a cutoff before it is turned into containers, or its idle cost is mostly an artifact. Simple copies and Holt–Winters are hard to beat on error, and a high quantile is how any forecaster buys a lower shortfall. A choice of forecaster per class, made on two days, did not beat one simple forecaster on the held-out days, and a reactive keep-alive reaches a lower shortfall than an hourly forecast.

II. Related Work

A. Workload Characterization and Keep-Alive Strategies

A prerequisite for effective serverless resource allocation is understanding real-world invocation patterns. In their seminal characterization of Microsoft Azure Functions, Shahrad et al. demonstrated that serverless workloads exhibit extreme skewness: the vast majority of functions are invoked infrequently, inter-arrival times span several orders of magnitude, and memory consumption remains concentrated in a small subset

xxxx-..-....-....-x/xx/$xx.00 ©2026IEEE

of applications [1]. To curb cold starts while limiting idle resource consumption, they proposed a practical hybrid keep-alive policy that tracks idle-time histograms and falls back to a time-series predictor when inter-invocation intervals exceed observed thresholds. Building upon these observations, recent efforts such as Lambda-Warmer [3] employ measurement-driven adaptation to tune keep-alive intervals dynamically based on empirical container reuse patterns rather than static timeouts. Keep-alive rules are reactive, which motivates proactive invocation forecasting.

B. Predictive Autoscaling and Proactive Schedulers

To anticipate arrival surges before cold starts occur, recent research integrates predictive modeling directly into the serverless autoscaling loop. Rather than assessing forecasting models in isolation, modern frameworks couple predictions with custom scaling controllers. For instance, Zhang et al. introduced AAPA [4], an archetype-aware predictive autoscaler on Kubernetes that decomposes invocation histories into spike, periodic, ramp, and stationary patterns, mapping each archetype to a tailored autoscaling policy with uncertainty quantification. Daraghmeh et al. framed container scaling as a classification problem, leveraging machine learning classifiers to determine whether to load or unload containers based on historical feature engineering [5].

Beyond pure classification, hybrid and control-theoretic methodologies have gained traction. Shiekhani et al. combined ARIMA with gradient-boosted decision trees to pre-warm containers and schedule functions on OpenWhisk [6], while Nguyen et al. integrated Fourier series forecasting with model predictive control (MPC) to guarantee proactive container readiness under latency constraints [7]. Similarly, deep neural architectures have been deployed to predict cold-start events directly, subsequently modulating provisioned concurrency levels [8]. These studies evaluate whole controllers, where forecast quality is entangled with scheduling and container reuse. Our study isolates the forecaster under one protocol and asks how its predictions translate into container gaps.

C. Time-Series Forecasters: From Heuristics to Foundation Models

The landscape of time-series forecasting in systems research spans a spectrum from lightweight empirical heuristics to sophisticated neural architectures. For periodic and predictable workloads, domain-specific baselines such as seasonal naive repetition and classical exponential smoothing (e.g., Holt–Winters ETS) remain widely used due to their minimal computational overhead and interpretability. For complex non-linear patterns, deep recurrent models such as Long Short-Term Memory (LSTM) networks have long represented the standard learned approach, though they require extensive historical data and specialized training pipelines.

Most recently, the emergence of pretrained time-series foundation models has challenged the convention of training task-specific predictors. Ansari et al. introduced Chronos [2], a probabilistic framework that tokenizes real-valued time series and trains language-model architectures across massive, heterogeneous corpora to achieve zero-shot forecasting on unseen domains. Its high-throughput variant, Chronos-Bolt, emits multi-horizon quantile forecasts in a single forward pass, while Chronos-2 extends these capabilities to multivariate settings with exogenous covariates [9]. In serverless provisioning, upper quantiles offer an attractive mathematical mechanism to hedge against cold-start shortfalls by deliberately pre-warming surplus capacity. The comparison below asks whether that zero-shot forecast beats the simple baselines on error, and what idle capacity it spends to cut the shortfall.

III. Setup

The trace records invocations per minute for 14 days [1]. Class statistics use days 1–10 only. A missing day is treated as 1,440 zeros. A function is sparse when at least 90% of those minutes are zero. Otherwise it is daily when the autocorrelation at a lag of one day exceeds 0.30. A remaining function is bursty when its coefficient of variation is at least 2.05, the 75th percentile among non-sparse functions. The rest are labeled other. We draw 20 eligible functions from each class with seed 42. A function is eligible when it is invoked at least once in days 1–10 and at least once in days 11–14. The trace contains 68,394 functions in the first ten days, of which 54,502 are sparse.

Each forecast predicts the next 60 minutes. A new forecast starts every 60 minutes across days 11–14, so the 96 windows cover those four days without overlap. Days 1–10 are history. Days 11–14 are the test. Days 13–14 are also reported alone, as a check that a choice made on days 11–12 still holds. Seasonal Holt–Winters, Chronos, and the LSTM each see the previous 1,440 minutes, one full day. Last-value and the trailing percentile use only the latest minutes, so a longer history would not change them.

Six forecasters form the main comparison. Seasonal naive repeats the same hour of the previous day when the training day-lag autocorrelation exceeds 0.30, and otherwise repeats the previous 60 minutes. Last-value repeats the latest observed minute. The trailing P95 holds the 95th percentile of the previous 10 minutes for the whole hour. ETS is additive Holt–Winters with a daily season. Its smoothing weights are chosen on days 1–10 only, separately for each function, on a grid of level, trend, and seasonal coefficients, by one-step squared error. The chosen weights then filter the series, and a forecast uses only the state from minutes before its origin, so ETS carries the whole history through its state. Chronos is the median forecast of `amazon/chronos-bolt-small`, applied with no training on this trace. It has no separate paper, so we cite the Chronos paper and name the checkpoint. The LSTM is one shared network with a single layer of 32 units and a linear output layer. It is trained on windows from days 1–8 with absolute error, early-stopped on days 9–10, and each window is scaled by the mean of its own context. A first version used a ReLU output layer. Many of its 60 output units died and predicted zero in every window, so it was replaced.

Further forecasters serve as references. Previous hour repeats the previous 60 minutes for every function. Always-previous-day repeats the previous day for every function, which removes the autocorrelation gate. TSB and Croston are intermittent-demand forecasters with smoothing 0.1, fitted on the previous day. An LSTM trained with the pinball loss at 0.9 and the Chronos 80th and 90th percentiles are high-forecast variants. Margin variants multiply a forecast by 1.25, 1.5, or 2. A reactive keep-alive is a reference of a different kind: at every minute it keeps warm the largest container count used in the previous 10 minutes. It reads actual counts each minute, so it is not a 60-minute forecast.

Forecasts below 0.5 invocations are set to zero for every model before scoring. Otherwise the ceiling in the score gives a forecast of 0.01 a whole container.

The provisioning score assumes that one warm container serves 10 invocations in a minute. For a forecast $\hat{y}_t$ of the actual count y_t,

$$S = \sum_t \max(0,\, \lceil y_t/10 \rceil - \lceil \hat{y}_t/10 \rceil)\,, \tag{1}$$

$$O = \sum_t \max(0,\, \lceil \hat{y}_t/10 \rceil - \lceil y_t/10 \rceil)\,. \tag{2}$$

S counts short container-minutes and is the stand-in for cold starts: the trace has no container-start events, so a short minute means the forecast prepared fewer containers than the count required. O counts extra container-minutes, the idle capacity. Idle cost weights O by memory. For each function we take its application's median allocated memory on days 1–10, using the 50th percentile published with the trace. All 80 functions have that value. The median across functions is 144 MB, and the range is 90 to 518 MB. Extra megabyte-minutes are O times that memory. The trace reports memory per application, and three applications contribute two functions each, so the weight is a size rather than a bill. No price is applied. Forecasts are clipped at zero before either sum, so a negative prediction prepares no containers. A forecast error that stays inside the same container adds nothing to either sum: eight invocations against a forecast of five still needs one container. Any forecast in $(0, 10]$ prepares one container, so a small positive value on an idle minute costs a full container-minute. Forecast error is the symmetric mean absolute percentage error. For one function it is the mean over test minutes of

$$\mathrm{sMAPE}_t = 100 \cdot \frac{2|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|}, \tag{3}$$

and a minute where both values are zero contributes nothing. Tables report the median of that per-function mean. A few heavy functions dominate class sums, which is why the median is the summary. A third count, new containers, is $\sum_t \max(0,\, r_t - \max(p_t, r_{t-1}))$ with $r_t = \lceil y_t/10 \rceil$ required and $p_t = \lceil \hat{y}_t/10 \rceil$ prepared. It counts a container that must be created because the warm set, taken to be the larger of the prepared and the previously used containers, is too small, so a long run of short minutes counts once per added container and not once per minute. Intervals are 95% percentile bootstrap intervals over functions, resampled within class, from 2,000 draws. Paired differences use the same draws.

IV. Evaluation

A. Scoring cutoff

A first scoring of the same forecasts gave ETS, the LSTM, and Chronos median extra capacities of 2,101.5, 3,640.5, and 3,281.0 container-minutes. Those values came from the ceiling. On the sparse functions the Chronos forecast was positive on 88% of minutes and below 0.5 on almost all of them, and ETS was positive on 50%. Each such minute prepared a whole container. After the cutoff the same medians are 112.5, 0.5, and 1.5. Every number below uses the cutoff for every model. Fixed forecasts such as seasonal naive and the trailing percentile are barely affected.

B. Error

Table I summarizes days 11–14. Previous hour has the lowest overall median sMAPE, 4.7%. Seasonal naive has 7.1% and ETS 8.2%, and the paired median difference between ETS and seasonal naive is 0.0 points per function (95% interval -0.0 to 0.7). Chronos has 13.5%, the LSTM 17.0%, last-value 19.2%, and the trailing P95 24.7%. The intervals are wide. Seasonal naive has 7.1% with an interval of 2.7 to 15.3, so differences of a few points among the first three are within noise. The class split shows who is good where. ETS is best on daily functions, 2.1% against 3.9% for seasonal naive. Chronos is best on other functions, 7.8% against 12.9%, and Chronos and the LSTM are slightly ahead of seasonal naive on bursty functions, 34.8% and 34.6% against 41.0%. Chronos and the LSTM tie on sparse functions at 0.7%, because both predict almost nothing. The autocorrelation gate in seasonal naive does not matter overall: always repeating the previous day gives 9.6% and always repeating the previous hour 4.7%, so on these minute counts yesterday's shape is not what makes the copy accurate. Fig. 1 shows one daily and one sparse function.

C. Shortfall and idle capacity

Every forecaster is short on most functions (Table II): from 66% of functions for the trailing P95 to 90% for previous hour. Seasonal naive has a median shortfall of 75 container-minutes (interval 29 to 169) and 80.5 extra. The trailing P95 has a median shortfall of 18.5 (4 to 52) and 729 extra (369 to 2,177). The paired difference to seasonal naive is -35.5 short container-minutes (-108.5 to -15.0) and $+426.5$ extra (214 to 1,228). New containers fall from 25.5 to 12.5. In application memory, the extra capacity grows from 15.3 to 135.4 thousand megabyte-minutes. Chronos at its 90th percentile gives almost the same point: 18.5 short, 754.5 extra, 8.5 new containers. ETS (52.5, 112.5) has a somewhat lower shortfall than seasonal naive and somewhat more idle capacity. Chronos median (253.5, 1.5) and the LSTM (303.5, 0.5) trade the other way: they leave little idle capacity and are short on more minutes.

Fig. 2 gives headroom to each forecaster and pools functions, so heavy functions count in proportion to their load.

The curves are close. Moving up a quantile of recent load or of Chronos buys shortfall at a slightly lower price than multiplying seasonal naive by 1.25 or 1.5. No amount of headroom brings the pooled shortfall below about 14% of the required container-minutes, because the bursty functions dominate it. An LSTM trained to a 0.9 level, one run, lands near the same curve with more idle capacity: 20.5 short and 1,205.5 extra. The reactive 10-minute keep-alive is far below every curve: a median of 6 short container-minutes and 1,038 extra, with a pooled shortfall of 1.6% at 43% extra. It reads the actual counts each minute, so it is not a competing forecaster. It shows what an hourly forecast interval gives up.

D. Class-specific choices and the test days

Table III compares single forecasters with two class-specific picks. A pick takes, for each class, the forecaster that is best on days 11–12. One pick ranks by shortfall and then by idle capacity. The other ranks by short plus extra container-minutes. Days 13 and 14 are a weekend: the sampled functions have 1.23 and 0.99 million invocations, against 3.41 and 3.06 million on days 11 and 12, and the same drop appears on days 6 and 7. The medians for days 13 and 14 are therefore lower for every forecaster and are not like-for-like with days 11 and 12. The shortfall-first pick chooses ETS for sparse and the trailing P95 for the other three classes, and on days 13 and 14 it matches always using the trailing P95. The short-plus-extra pick chooses the LSTM for sparse, ETS for daily, Chronos for bursty, and the trailing P95 for other functions. It has a median of 5 short and 0 extra on days 13 and 14, against 12 and 15 for seasonal naive, but the paired median differences are 0 short (interval 0 to 0) and 0 extra (-2.0 to 0.0), so the gain is not distinguishable from zero. Twenty functions per class and two selection days are too little to support a different forecaster for each class.

E. Capacity and weighting

A fixed capacity of 10 invocations per container-minute is arbitrary. The median average duration of the sampled functions is 326 ms, so one sequential container could serve about 184 invocations a minute. Capacity from each function's own average duration, with a floor of one, gives for seasonal naive 32.5 short and 38.0 extra, for the trailing P95 5.0 and 246.0, for ETS 29.0 and 34.5, for Chronos 69.0 and 0.0, and for the LSTM 88.5 and 0.0. The ordering is the same as at 10. Weighting each function by the number of eligible functions in its class, so that the sparse class counts for most of the fleet, gives medians of 17 short and 17 extra for seasonal naive, 15 and 129 for the trailing P95, 16 and 12 for ETS, and 24 and 0 for Chronos. The All column in the tables is a statistic of the sample, not of the fleet.

F. Limits

The trace is from 2019 and contributes 80 sampled functions, 20 per class, not a live deployment. The class thresholds were not varied. Eligibility required at least one invocation on days 11–14, so a function that goes silent in the test window never enters the sample. The counts are per minute, so S is a capacity gap, and new containers is a proxy for cold starts, not a measurement on a platform. Containers are counted per function with one invocation at a time, while Azure scales per application. Memory is the allocated size of the application. The forecasts are issued once an hour and held, which favours methods that copy the past hour or day and penalizes reactive ones. Only the fixed 10-minute keep-alive is evaluated, not the histogram policy of Shahrad et al. [1]. The cutoff of 0.5 was set once, before scoring, and applied to all models. The LSTM is one run of 12 epochs whose validation loss was still falling, and no per-class learned model was trained.

TABLE I
MEDIAN SMAPE, SHORT CONTAINER-MINUTES, AND EXTRA CONTAINER-MINUTES ON DAYS 11–14.

Model	Metric	Sparse	Daily	Bursty	Other	All
Seasonal naive	sMAPE (%)	0.9	3.9	41.0	12.9	7.1
	Short	13.0	45.0	718.0	150.0	75.0
	Extra	13.0	43.0	719.0	150.5	80.5
Previous hour	sMAPE (%)	1.1	3.3	41.0	12.9	4.7
	Short	15.5	37.0	718.0	150.0	66.0
	Extra	15.5	43.0	719.0	150.5	69.5
Last value	sMAPE (%)	0.9	37.8	48.2	11.9	19.2
	Short	17.0	1,150	763.5	99.5	410.0
	Extra	0.0	31.0	593.5	93.5	63.5
Trailing P95	sMAPE (%)	3.8	134.4	74.6	12.9	24.7
	Short	15.0	0.0	143.0	0.0	18.5
	Extra	87.5	4,386	2,183	98.0	729.0
ETS	sMAPE (%)	0.9	2.1	55.1	13.7	8.2
	Short	13.0	45.0	615.5	47.0	52.5
	Extra	9.5	8.0	969.5	166.5	112.5
LSTM	sMAPE (%)	0.7	39.9	34.6	12.0	17.0
	Short	19.0	1,150	924.0	0.0	303.5
	Extra	0.0	0.0	125.5	142.5	0.5
Chronos	sMAPE (%)	0.7	34.3	34.8	7.8	13.5
	Short	19.0	1,088	996.5	0.0	253.5
	Extra	0.0	0.0	48.0	144.0	1.5

TABLE II
ALL 80 FUNCTIONS, DAYS 11–14. MEDIANS. NEW IS NEW CONTAINERS, MB IS THOUSANDS OF EXTRA MEGABYTE-MINUTES, AND SHORT% IS THE SHARE OF FUNCTIONS WITH ANY SHORTFALL. THE STAR MARKS A REACTIVE REFERENCE THAT READS ACTUAL COUNTS.

Model	sMAPE	Short	Extra	New	MB	Short%
Seasonal naive	7.1	75.0	80.5	25.5	15.3	88
Previous hour	4.7	66.0	69.5	25.0	12.6	90
Last value	19.2	410.0	63.5	141.5	12.4	88
Trailing P95	24.7	18.5	729.0	12.5	135.4	66
ETS	8.2	52.5	112.5	21.0	17.0	79
LSTM	17.0	303.5	0.5	141.5	0.1	84
LSTM, tau 0.9	35.3	20.5	1,206	6.0	194.6	68
Chronos	13.5	253.5	1.5	127.0	0.4	85
Chronos q90	22.9	18.5	754.5	8.5	111.2	68
TSB	17.3	224.0	10.0	111.0	1.4	84
Croston	32.5	233.0	10.0	111.0	1.4	81
Keep-alive 10 min*	–	6.0	1,038	6.0	157.2	69

V. CONCLUSION

The goal is fewer cold starts. This trace cannot count them, so the evidence is the container shortfall and the idle capacity under a stated capacity, with idle capacity also weighted by application memory. Three findings stand. First, the idle cost of continuous forecasts depended on a cutoff for tiny forecasts; with it, ETS, Chronos, and an LSTM are not wasteful. Second,

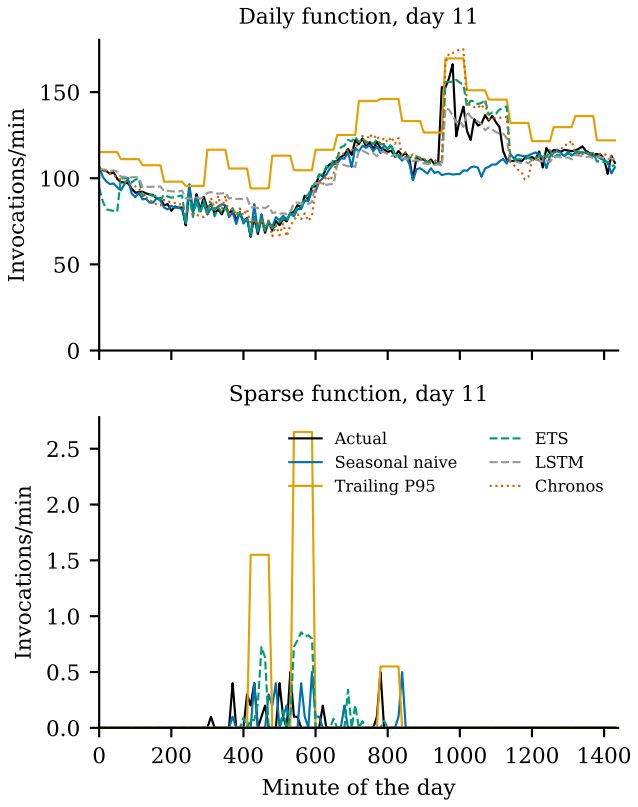


Fig. 1. Day 11 for one daily and one sparse function, in 10-minute means. The other forecasters follow the same shape or stay near zero.

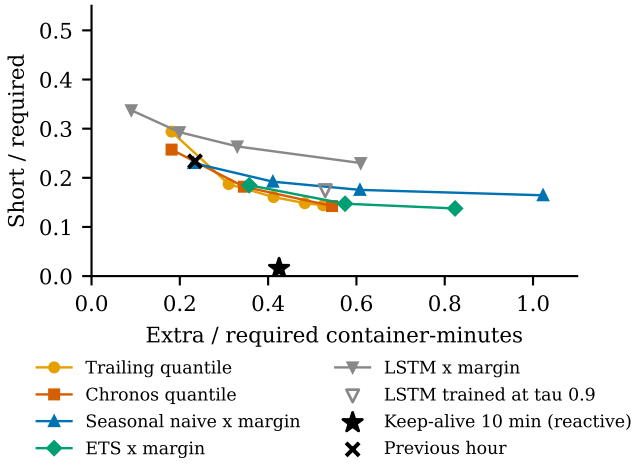


Fig. 2. Pooled short and extra container-minutes as a share of the required container-minutes on days 11–14, with headroom added to each forecaster. Closer to the lower left is better.

a previous-hour copy, seasonal naive, and Holt–Winters are the most accurate and statistically close, and neither Chronos nor the LSTM beats them overall, although Chronos is best on other functions and ETS on daily ones. Third, shortfall is bought with headroom, and a high quantile of recent load

TABLE III
MEDIANS FOR ALL FUNCTIONS ON DAYS 11–12 AND ON THE WEEKEND DAYS 13–14. THE TWO PICKS CHOOSE A FORECASTER PER CLASS ON DAYS 11–12.

	Days 11–12			Days 13–14		
Policy	**sMAPE**	**Short**	**Extra**	**sMAPE**	**Short**	**Extra**
Seasonal naive	8.4	48.5	59.0	2.7	12.0	15.0
Trailing P95	32.6	13.0	694.5	16.2	0.5	120.5
ETS	9.4	30.5	55.5	7.8	8.5	39.5
LSTM	21.2	271.5	0.5	12.3	70.5	0.0
Chronos	15.7	199.5	1.0	7.6	47.0	0.0
Pick: shortfall first	26.4	12.5	694.5	12.7	0.0	120.0
Pick: short plus extra	7.8	37.5	2.0	1.6	5.0	0.0

buys it at about the same price as a high quantile of Chronos: a median shortfall of 18.5 container-minutes instead of 75, for about 730 extra. A class-specific choice made on two days was not distinguishable from seasonal naive, and a reactive 10-minute keep-alive had a far lower shortfall than any hourly forecast. The next measurement is shorter forecast intervals with rolling re-forecasting, and cold starts recorded on a real platform.

REFERENCES

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 205–218.

[2] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado *et al.*, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=gerNCVqqtR

[3] Y. Singh and R. Shrestha, "Lambda-warmer: A measurement-driven adaptive framework for cost-aware cold-start mitigation in AWS lambda," *Future Generation Computer Systems*, vol. 185, p. 108681, 2026.

[4] G. Zhang, S. Vippagunta, R. Nandagopal, S. Raman, J. Xu, M. Pfeiffer *et al.*, "AAPA: An archetype-aware predictive autoscaler with uncertainty quantification for serverless workloads on Kubernetes," *arXiv preprint arXiv:2507.05653*, 2025.

[5] M. Daraghmeh, Y. Jararweh, and A. Agarwal, "Leveraging machine learning and feature engineering for optimal data-driven scaling decision in serverless computing," *Simulation Modelling Practice and Theory*, vol. 140, p. 103090, 2025.

[6] L. Shiekhani, H. Wang, W. Shi, J. Liu, Y. Qiu, C. Gu, and W. Ding, "The hybrid model: Prediction-based scheduling and efficient resource management in a serverless environment," *Applied Sciences*, vol. 15, no. 14, p. 7632, 2025.

[7] C. Nguyen, M. Bhuyan, and E. Elmroth, "Taming cold starts: Proactive serverless scheduling with model predictive control," *arXiv preprint arXiv:2508.07640*, 2025.

[8] N. S. Kumar and S. S. Samy, "Cold start prediction and provisioning optimization in serverless computing using deep learning," *Concurrency and Computation: Practice and Experience*, 2025.

[9] A. F. Ansari, O. Shchur, J. Küken, A. Auer, B. Han, P. Mercado *et al.*, "Chronos-2: From univariate to universal forecasting," *arXiv preprint arXiv:2510.15821*, 2025.